YouNICon | Video performance, engagement, toxicity & audiences

An integrated, reproducible analysis of manually labeled conspiracy and non-conspiracy videos

Research sequence: video performance → publication-year adjustment → engagement rates → within-channel comparisons → comment toxicity → audience overlap → cross-content toxicity.

Scope: manually labeled videos (`majority_label` = 0 or 1), not predictions from a separate classifier.

1 | Environment, data paths & figure design

Run from your Princeton Della Jupyter environment. The notebook reads the original CSVs and does not modify them. Figures are displayed inline; optional saving is disabled by default.

In [1]:
```
from pathlib import Path
import warnings, platform
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import FuncFormatter, PercentFormatter
from IPython.display import display
from scipy.stats import mannwhitneyu
import statsmodels.formula.api as smf

DATA_DIR = Path("/home/rg6905/YouNiCon")
LABELS_FILE = DATA_DIR / "conspiracy_label.csv"
VIDEOS_FILE = DATA_DIR / "all_videos.csv"
COMMENTS_FILE = DATA_DIR / "comments_anon.csv"
CHUNK_SIZE = 500_000
SAVE_FIGURES = False  # set True to save figures in a new folder
FIG_DIR = DATA_DIR / "research_figures"
if SAVE_FIGURES:
    FIG_DIR.mkdir(parents=True, exist_ok=True)

for p in (LABELS_FILE, VIDEOS_FILE, COMMENTS_FILE):
    if not p.is_file():
        raise FileNotFoundError(f"Required input not found: {p}")

COLORS = {"Non-conspiracy": "#3877A8", "Conspiracy": "#D76A50", "Both types": "#7A61A8"}
sns.set_theme(style="whitegrid", context="talk", font_scale=.83,
              rc={"axes.spines.top": False, "axes.spines.right": False,
                  "grid.alpha": .18, "figure.facecolor": "white",
                  "axes.titleweight": "bold"})
plt.rcParams["figure.dpi"] = 120

def finish(fig, name):
    fig.tight_layout()
    if SAVE_FIGURES:
        fig.savefig(FIG_DIR / f"{name}.png", dpi=220, bbox_inches="tight")
    plt.show()

def grouped_plot(data, metric, title, ylabel, log_axis=False):
    x = data[["content_type", metric]].dropna()
    if x.empty:
        print(f"No valid observations for {metric}")
        return
    fig, ax = plt.subplots(figsize=(9.4, 5.4))
    sns.boxplot(data=x, x="content_type", y=metric,
                order=["Non-conspiracy", "Conspiracy"],
                palette=[COLORS["Non-conspiracy"], COLORS["Conspiracy"]],
                showfliers=False, width=.52, ax=ax)
    if log_axis:
        ax.set_yscale("symlog", linthresh=1)
    ax.set(title=title, xlabel="", ylabel=ylabel)
    finish(fig, metric + "_distribution")

print("Python:", platform.python_version(), "| pandas:", pd.__version__)
```

```
Python: 3.13.3 | pandas: 2.2.3
```

2 | Load labels and metadata

We read only the columns required for analysis and scan the large video CSV in chunks. Duplicate or conflicting labels are audited rather than silently accepted.

In [2]:
```
labels = pd.read_csv(LABELS_FILE, usecols=["video_id", "majority_label"],
                     dtype={"video_id": "string"})
labels["majority_label"] = pd.to_numeric(labels["majority_label"], errors="coerce")
labels = labels.dropna(subset=["video_id"])
conflicts = labels.groupby("video_id")["majority_label"].nunique()
if (conflicts > 1).any():
    raise ValueError(f"Conflicting labels for {(conflicts > 1).sum()} videos")
labels = labels.drop_duplicates("video_id")
labels = labels[labels["majority_label"].isin([0, 1])].copy()
labels["majority_label"] = labels["majority_label"].astype(int)
display(labels["majority_label"].value_counts().sort_index().rename(index={0:"Non-conspiracy",1:"Conspiracy"}).to_frame("Videos"))

wanted = {"video_id", "Channel_ID", "Published_date", "Views", "Likes",
          "NumComments", "DurationSec"}
found = []
seen_rows = 0
ids = set(labels["video_id"].astype(str))
for chunk in pd.read_csv(VIDEOS_FILE, chunksize=CHUNK_SIZE,
                         usecols=lambda c: c in wanted,
                         dtype={"video_id": "string", "Channel_ID": "string"},
                         low_memory=False):
    seen_rows += len(chunk)
    hit = chunk[chunk["video_id"].isin(ids)]
    if not hit.empty:
        found.append(hit.copy())
if not found:
    raise ValueError("No labeled videos matched all_videos.csv. Check paths and video ID formats.")
metadata = pd.concat(found, ignore_index=True)
print(f"Scanned {seen_rows:,} metadata rows; matched {len(metadata):,} rows.")
print("Duplicate matched video rows:", int(metadata["video_id"].duplicated().sum()))
metadata = metadata.drop_duplicates("video_id")
df = labels.merge(metadata, on="video_id", how="left", validate="one_to_one", indicator=True)
df["metadata_found"] = df["_merge"].eq("both")
df = df.drop(columns="_merge")
print("Labeled videos missing metadata:", int((~df["metadata_found"]).sum()))
display(df.head(3))
```

majority_label	Videos
Non-conspiracy	2017
Conspiracy	1144

```
Scanned 596,967 metadata rows; matched 3,160 rows.
Duplicate matched video rows: 0
Labeled videos missing metadata: 1
```

	video_id	majority_label	Channel_ID	Published_date	Views	Likes	NumComments	DurationSec	metadata_found
0	-3pY4UfY-Mw	1	UC_KJtBlAAqrd6R04YQry5-w	2013-11-16 13:24:25	154.0	2.0	13.0	505.0	True
1	-7-B4aGqrCQ	1	UCQwCjFkhLbdpsu1xUDm7oGA	2015-06-10 04:43:07	1748.0	24.0	2.0	1081.0	True
2	-8u-hsE87xw	1	UC6804ISS_CniXpgRzm35OpA	2019-04-24 06:58:18	8313.0	777.0	128.0	1960.0	True

In [3]:
```
for c in ["Views", "Likes", "NumComments", "DurationSec"]:
    if c not in df:
        df[c] = np.nan
    df[c] = pd.to_numeric(df[c], errors="coerce")
    df.loc[df[c] < 0, c] = np.nan
if "Published_date" not in df:
    df["Published_date"] = pd.NaT
df["Published_date"] = pd.to_datetime(df["Published_date"], errors="coerce", utc=True)
df["publish_year"] = df["Published_date"].dt.year
df["content_type"] = df["majority_label"].map({0:"Non-conspiracy", 1:"Conspiracy"})
df["log1p_views"] = np.log1p(df["Views"])
df["log1p_duration"] = np.log1p(df["DurationSec"])
```

```
df["likes_per_1000_views"] = np.where(df["Views"] > 0, 1000 * df["Likes"] / df["Views"], np.nan)
df["comments_per_1000_views"] = np.where(df["Views"] > 0, 1000 * df["NumComments"] / df["Views"], np.nan)

audit = df.groupby("content_type", observed=True).agg(
    labeled_videos=("video_id", "size"),
    metadata_found=("metadata_found", "sum"),
    valid_views=("Views", "count"),
    valid_year=("publish_year", "count"),
    channels=("Channel_ID", "nunique")
)
display(audit)
display(df[["Views","Likes","NumComments","Published_date","Channel_ID"]].isna().sum().to_frame("Missing values"))
```

content_type	labeled_videos	metadata_found	valid_views	valid_year	channels
Conspiracy	1144	1144	1144	1144	436
Non-conspiracy	2017	2016	2015	2016	616

	Missing values
Views	2
Likes	190
NumComments	212
Published_date	1
Channel_ID	1

3 | Views: descriptive performance

Views are highly skewed. Report medians and interquartile ranges as well as log-transformed distributions. These are **not** age-adjusted comparisons.

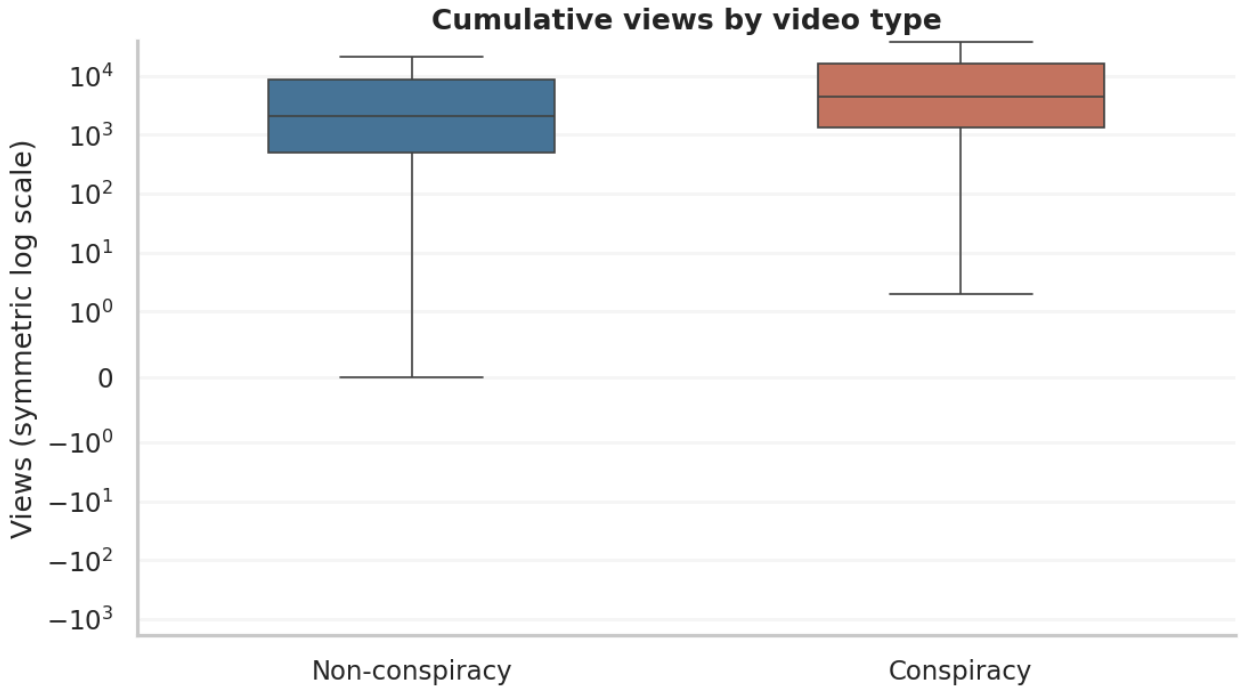
In [5]:
```
view_summary = df.groupby("content_type")[["Views", "log1p_views"]].agg(["count","median","mean"])
display(view_summary.round(2))
grouped_plot(df, "Views", "Cumulative views by video type", "Views (symmetric log scale)", log_axis=True)
fig, ax = plt.subplots(figsize=(9,5))
sns.ecdfplot(data=df.dropna(subset=["log1p_views"]), x="log1p_views",
             hue="content_type", hue_order=["Non-conspiracy","Conspiracy"],
             palette=COLORS, linewidth=2.5, ax=ax)
ax.set(xlabel="log(1 + cumulative views)", ylabel="Cumulative share of videos",
       title="Distribution of video reach")
finish(fig, "views_ecdf")
```

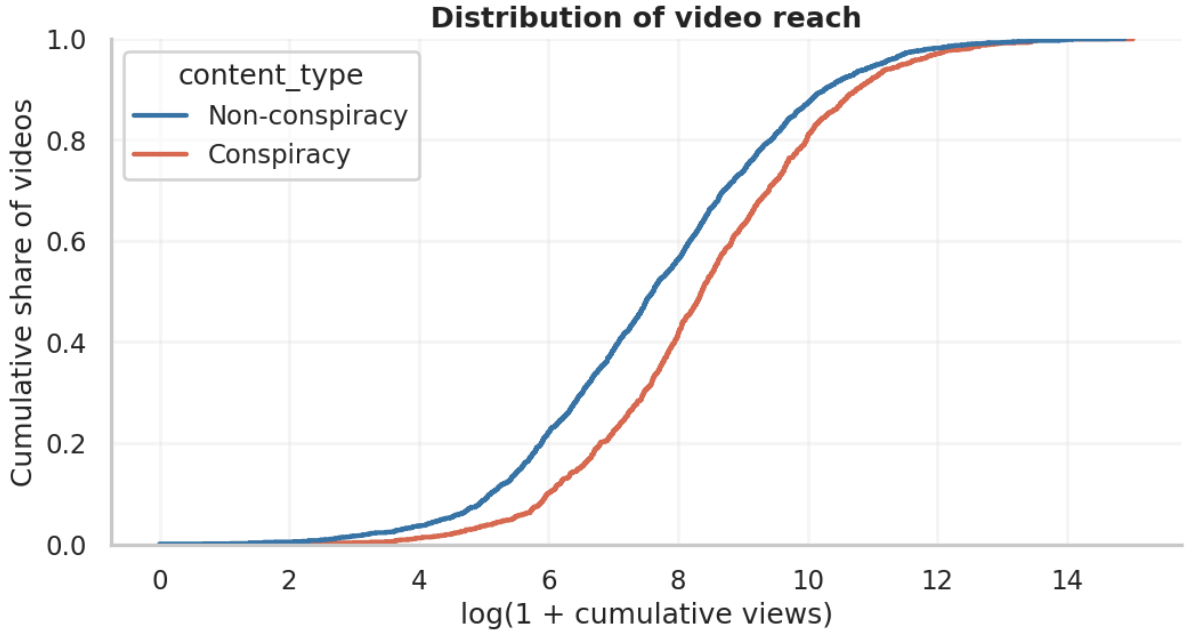
	Views			log1p_views		
	count	median	mean	count	median	mean
content_type						
Conspiracy	1144	4316.5	27244.8	1144	8.37	8.40
Non-conspiracy	2015	2022.0	20030.6	2015	7.61	7.66

```
/tmp/ipykernel_3260809/2062234863.py:45: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=x, x="content_type", y=metric,
```


Cumulative views by video type
Views (symmetric log scale)
10^4
10^3
10^2
10^1
10^0
0
-10^0
-10^1
-10^2
-10^3
Non-conspiracy
Conspiracy

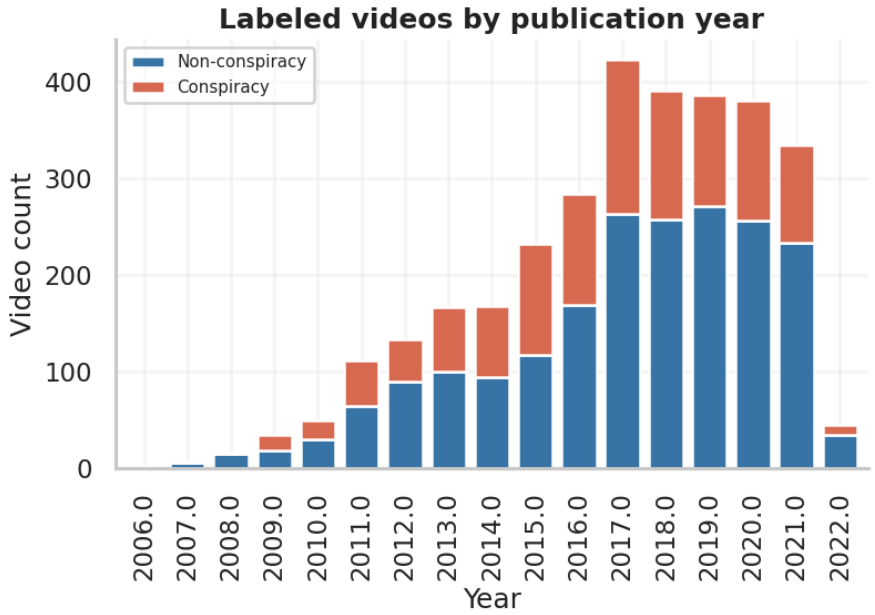

Distribution of video reach
content_type
Non-conspiracy
Conspiracy
Cumulative share of videos
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8
10
12
14
log(1 + cumulative views)

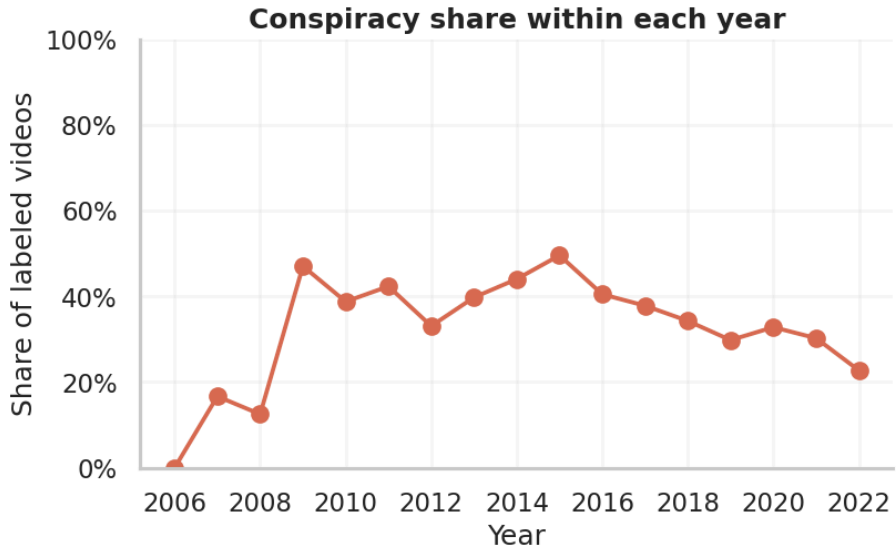
Publication year is a proxy for exposure vintage, **not a substitute for a known observation date**. We compare within publication years and estimate year fixed-effects associations. Interpret these as descriptive associations.

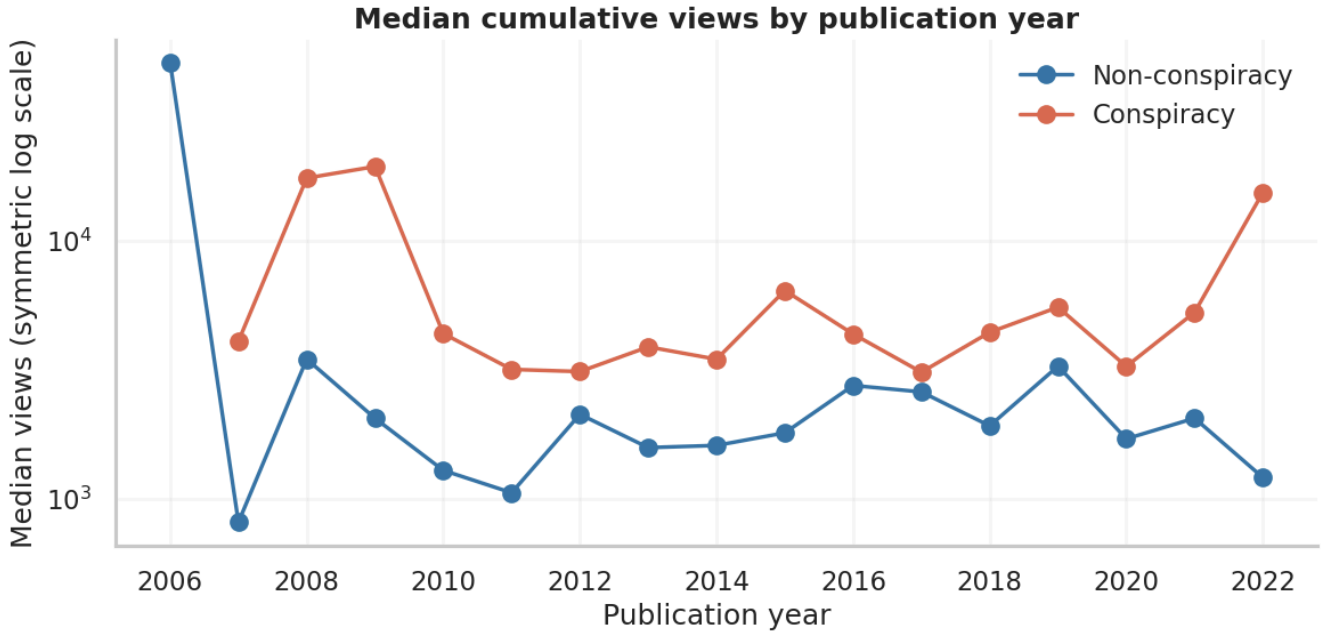
In [6]:
```
yearly = (df.dropna(subset=["publish_year"])
            .groupby(["publish_year", "content_type"]).size()
            .unstack(fill_value=0).reindex(columns=["Non-conspiracy","Conspiracy"], fill_value=0))
display(yearly.tail(20))
if not yearly.empty:
    fig, axes = plt.subplots(1, 2, figsize=(14,5))
    yearly.plot(kind="bar", stacked=True, color=[COLORS["Non-conspiracy"], COLORS["Conspiracy"]],
                ax=axes[0], width=.8)
    axes[0].set(title="Labeled videos by publication year", xlabel="Year", ylabel="Video count")
    axes[0].legend(title="", fontsize=9)
    shares = yearly.div(yearly.sum(axis=1), axis=0)
    shares["Conspiracy"].plot(marker="o", color=COLORS["Conspiracy"], ax=axes[1])
    axes[1].set(title="Conspiracy share within each year", xlabel="Year",
                ylabel="Share of labeled videos", ylim=(0,1))
    axes[1].yaxis.set_major_formatter(PercentFormatter(1))
    finish(fig, "year_composition")

year_medians = (df.dropna(subset=["publish_year","Views"])
    .groupby(["publish_year","content_type"])["Views"].median().unstack())
if not year_medians.empty:
    fig, ax = plt.subplots(figsize=(10,5))
    for typ in ["Non-conspiracy","Conspiracy"]:
        if typ in year_medians:
            ax.plot(year_medians.index, year_medians[typ], marker="o", lw=2,
                    color=COLORS[typ], label=typ)
    ax.set_yscale("symlog", linthresh=1)
    ax.set(title="Median cumulative views by publication year",
           xlabel="Publication year", ylabel="Median views (symmetric log scale)")
    ax.legend(frameon=False)
    finish(fig, "year_median_views")
```

content_type	Non-conspiracy	Conspiracy
publish_year		
2006.0	2	0
2007.0	5	1
2008.0	14	2
2009.0	18	16
2010.0	30	19
2011.0	64	47
2012.0	89	44
2013.0	100	66
2014.0	94	74
2015.0	117	115
2016.0	169	115
2017.0	263	160
2018.0	257	134
2019.0	271	115
2020.0	256	125
2021.0	233	101
2022.0	34	10







In [7]:
```python
def fit_adjusted(data, outcome, include_channel=False):
    cols = [outcome, "majority_label", "publish_year", "Channel_ID"]
    x = data[cols].dropna(subset=[outcome,"majority_label","publish_year"]).copy()
    x["publish_year"] = x["publish_year"].astype(int)
    if include_channel:
        x = x.dropna(subset=["Channel_ID"])
```

```
        mixed = x.groupby("Channel_ID")["majority_label"].nunique()
        x = x[x["Channel_ID"].isin(mixed[mixed == 2].index)].copy()
    if x["majority_label"].nunique() < 2 or len(x) < 20:
        print(f"Insufficient complete observations for {outcome}; within-channel={include_channel}")
        return None, x
    formula = outcome + " ~ majority_label + C(publish_year)"
    if include_channel:
        formula += " + C(Channel_ID)"
    try:
        if x["Channel_ID"].notna().sum() and x["Channel_ID"].nunique() >= 2:
            x = x.dropna(subset=["Channel_ID"])
            result = smf.ols(formula, data=x).fit(cov_type="cluster", cov_kwds={"groups":x["Channel_ID"]})
        else:
            result = smf.ols(formula, data=x).fit(cov_type="HC3")
        b = result.params.get("majority_label", np.nan)
        ci = result.conf_int().loc["majority_label"]
        print(f"{outcome} | within-channel={include_channel} | N={len(x):,} | "
              f"label coefficient={b:.4g} | 95% CI [{ci.iloc[0]:.4g}, {ci.iloc[1]:.4g}] | "
              f"p={result.pvalues['majority_label']:.3g}")
        return result, x
    except Exception as exc:
        print(f"Model could not be estimated: {exc}")
        return None, x

year_views_model, year_views_sample = fit_adjusted(df, "log1p_views")
```

```
log1p_views | within-channel=False | N=3,159 | label coefficient=0.7598 | 95% CI [0.4549, 1.065] | p=1.04e-06
```

```
/home/rg6905/.conda/envs/jup-env/lib/python3.13/site-packages/statsmodels/regression/linear_model.py:1884: RuntimeWarning: invalid value encountered in sqrt
  return np.sqrt(np.diag(self.cov_params()))
```

5 | Engagement rates

Likes and comments per 1,000 views are calculated only for videos with positive view counts. These rates are based on cumulative counts, not time-window engagement. Extreme ratios may arise from small denominators.

In [8]:
```
metrics = [
    ("likes_per_1000_views", "Likes per 1,000 views"),
    ("comments_per_1000_views", "Comments per 1,000 views"),
]
display(df.groupby("content_type")[[m for m,_ in metrics]].agg(["count","median","mean"]).round(3))
fig, axes = plt.subplots(1,2,figsize=(13.5,5.2))
for ax, (metric,label) in zip(axes, metrics):
    sub = df[["content_type",metric]].dropna()
    sns.boxplot(data=sub, x="content_type", y=metric,
                order=["Non-conspiracy","Conspiracy"],
                palette=[COLORS["Non-conspiracy"], COLORS["Conspiracy"]],
                showfliers=False, ax=ax)
    ax.set(xlabel="", ylabel=label, title=label + " by video type")
finish(fig, "engagement_rates")
for metric,_ in metrics:
    fit_adjusted(df, metric)
```

	likes_per_1000_views			comments_per_1000_views		
	count	median	mean	count	median	mean
content_type						
Conspiracy	1080	30.063	40.269	1082	7.282	11.750
Non-conspiracy	1889	36.364	45.608	1865	6.076	11.504

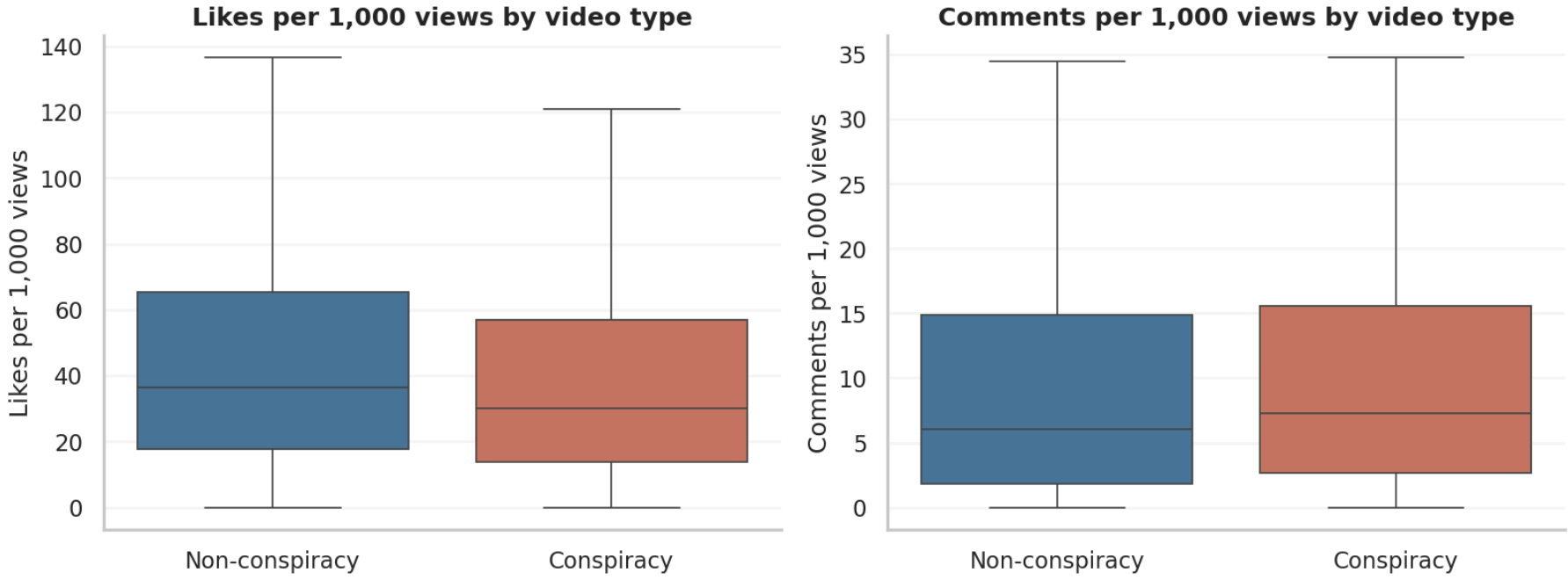
```
/tmp/ipykernel_3260809/1284893753.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=sub, x="content_type", y=metric,
/tmp/ipykernel_3260809/1284893753.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=sub, x="content_type", y=metric,
```




```
likes_per_1000_views | within-channel=False | N=2,969 | label coefficient=-0.9198 | 95% CI [-3.865, 2.025] | p=0.54
comments_per_1000_views | within-channel=False | N=2,947 | label coefficient=1.049 | 95% CI [-0.7128, 2.81] | p=0.243
/home/rg6905/.conda/envs/jup-env/lib/python3.13/site-packages/statsmodels/regression/linear_model.py:1884: RuntimeWarning: invalid value encountered in sqrt
  return np.sqrt(np.diag(self.cov_params()))
```

6 | Within-channel comparisons

Channel fixed effects use only channels containing **both** labels among complete observations. Year fixed effects are included. Standard errors are clustered by channel when possible. These models estimate associations, not causal effects.

In [10]:
```python
def fit_adjusted(data, outcome, include_channel=False):
    cols = [outcome, "majority_label", "publish_year", "Channel_ID"]

    x = data[cols].copy()

    # Convert pandas extension dtypes to statsmodels-compatible types
    x[outcome] = pd.to_numeric(x[outcome], errors="coerce")
    x["majority_label"] = pd.to_numeric(
        x["majority_label"], errors="coerce"
    )
    x["publish_year"] = pd.to_numeric(
        x["publish_year"], errors="coerce"
    )

    x = x.dropna(
        subset=[outcome, "majority_label", "publish_year"]
    )

    x["publish_year"] = x["publish_year"].astype(int)
    x["majority_label"] = x["majority_label"].astype(int)

    # Critical fix: convert Channel_ID to standard object dtype
    x["Channel_ID"] = x["Channel_ID"].astype(object)

    if include_channel:
        x = x.dropna(subset=["Channel_ID"])
```

```
        # Keep only channels containing both video categories
        mixed = x.groupby("Channel_ID")["majority_label"].nunique()

        x = x[
            x["Channel_ID"].isin(mixed[mixed == 2].index)
        ].copy()

    if x["majority_label"].nunique() < 2 or len(x) < 20:
        print(f"Insufficient observations for {outcome}")
        return None, x

    formula = outcome + " ~ majority_label + C(publish_year)"

    if include_channel:
        formula += " + C(Channel_ID)"

    try:
        if x["Channel_ID"].notna().all() and x["Channel_ID"].nunique() >= 2:
            model = smf.ols(formula, data=x).fit(
                cov_type="cluster",
                cov_kwds={"groups": x["Channel_ID"]}
            )
        else:
            model = smf.ols(formula, data=x).fit(cov_type="HC3")

        coefficient = model.params["majority_label"]
        ci = model.conf_int().loc["majority_label"]

        print(
            f"{outcome} | "
            f"within-channel={include_channel} | "
            f"N={len(x):,} | "
            f"channels={x['Channel_ID'].nunique()} | "
            f"coefficient={coefficient:.4f} | "
            f"95% CI [{ci.iloc[0]:.4f}, {ci.iloc[1]:.4f}] | "
            f"p={model.pvalues['majority_label']:.4g}"
        )

        return model, x

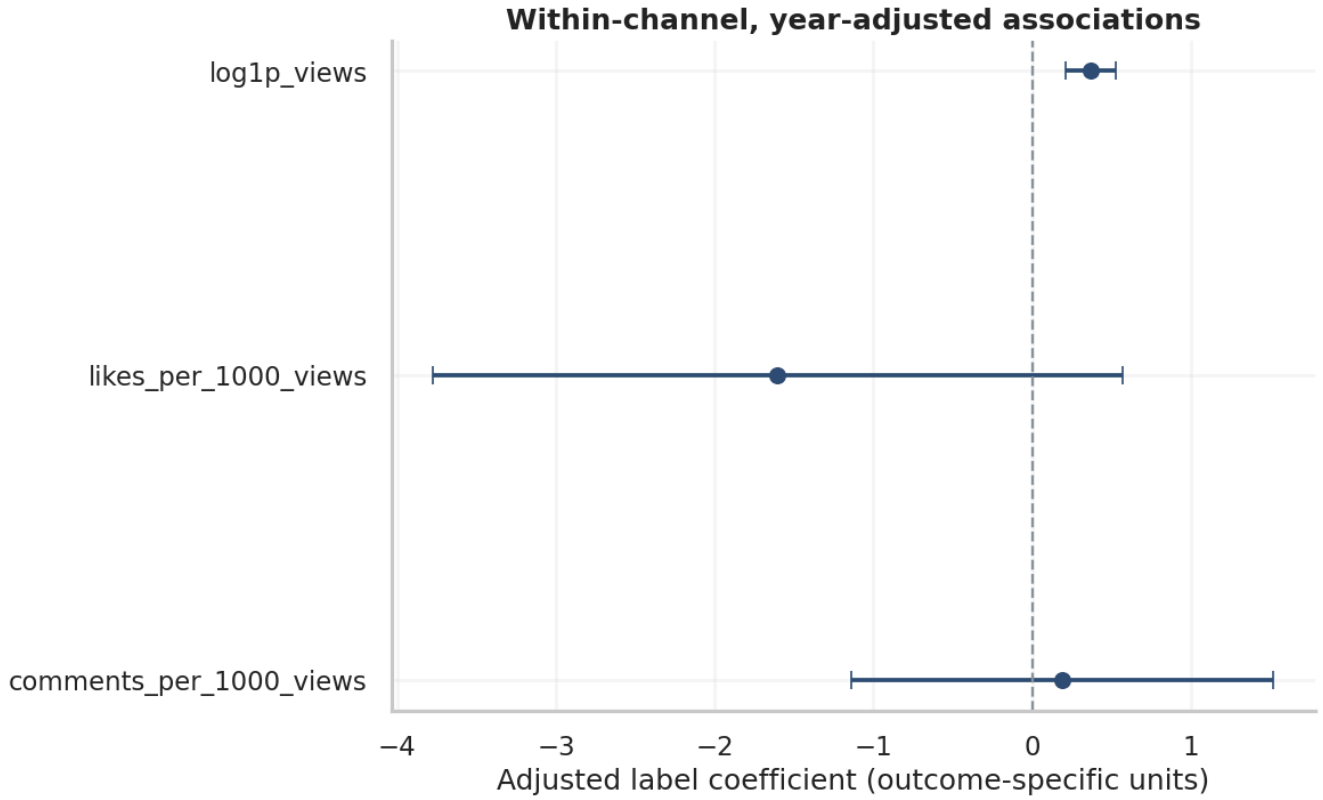
    except Exception as exc:
        print(f"Model could not be estimated: {exc}")
        return None, x

channel_models = {}
for outcome in ["log1p_views", "likes_per_1000_views", "comments_per_1000_views"]:
    model, sample = fit_adjusted(df, outcome, include_channel=True)
    channel_models[outcome] = (model, sample)

rows = []
for outcome, (model, sample) in channel_models.items():
    if model is None:
        continue
    ci = model.conf_int().loc["majority_label"]
    rows.append({"Outcome":outcome, "Coefficient":model.params["majority_label"],
                 "CI low":ci.iloc[0], "CI high":ci.iloc[1], "N":len(sample),
                 "Channels":sample["Channel_ID"].nunique()})
coef_table = pd.DataFrame(rows)
display(coef_table.round(4))
if not coef_table.empty:
    fig, ax = plt.subplots(figsize=(10, 1.2*len(coef_table)+2.6))
    y = np.arange(len(coef_table))
    ax.errorbar(coef_table["Coefficient"], y,
                xerr=[coef_table["Coefficient"]-coef_table["CI low"],
                      coef_table["CI high"]-coef_table["Coefficient"]],
                fmt="o", markersize=8, capsize=5, color="#304F73", ecolor="#304F73")
    ax.axvline(0, ls="--", color="#8A929E", lw=1.4)
    ax.set(yticks=y, yticklabels=coef_table["Outcome"], xlabel="Adjusted label coefficient (outcome-specific units)",
           title="Within-channel, year-adjusted associations")
    ax.invert_yaxis()
    finish(fig, "within_channel_coefficients")
```

```
log1p_views | within-channel=True | N=2,033 | channels=271 | coefficient=0.3705 | 95% CI [0.2144, 0.5266] | p=3.276e-06
likes_per_1000_views | within-channel=True | N=1,896 | channels=259 | coefficient=-1.6033 | 95% CI [-3.7736, 0.5671] | p=0.1477
comments_per_1000_views | within-channel=True | N=1,925 | channels=255 | coefficient=0.1910 | 95% CI [-1.1394, 1.5213] | p=0.7785
```

	Outcome	Coefficient	CI low	CI high	N	Channels
0	log1p_views	0.3705	0.2144	0.5266	2033	271
1	likes_per_1000_views	-1.6033	-3.7736	0.5671	1896	259
2	comments_per_1000_views	0.1910	-1.1394	1.5213	1925	255



7 | Comment toxicity by video category

The `toxic` column is treated as a numeric score. If it is binary, its mean is a toxicity rate; otherwise, it is a mean score. This section calculates both comment-weighted and video-weighted means, then compares yearly patterns. Only comments attached to labeled videos are included.

In [11]:
```
video_type = dict(zip(labels["video_id"].astype(str), labels["majority_label"]))
valid_ids = set(video_type)
video_stats = {}  # video_id -> [sum(toxic), count]
type_sum = {0:0.0, 1:0.0}
type_count = {0:0, 1:0}
toxic_min, toxic_max = np.inf, -np.inf
for chunk in pd.read_csv(COMMENTS_FILE, usecols=["videoId","toxic"],
                         dtype={"videoId":str}, chunksize=CHUNK_SIZE):
    chunk = chunk[chunk["videoId"].isin(valid_ids)].copy()
    chunk["toxic"] = pd.to_numeric(chunk["toxic"], errors="coerce")
    chunk = chunk.dropna(subset=["toxic"])
    if chunk.empty: continue
    toxic_min = min(toxic_min, chunk["toxic"].min())
```

```
    toxic_max = max(toxic_max, chunk["toxic"].max())
    chunk["label"] = chunk["videoId"].map(video_type)
    for label, part in chunk.groupby("label"):
        type_sum[int(label)] += float(part["toxic"].sum())
        type_count[int(label)] += len(part)
    for vid, part in chunk.groupby("videoId")["toxic"]:
        s, n = video_stats.get(vid, (0., 0))
        video_stats[vid] = (s+float(part.sum()), n+len(part))
print(f"Observed toxic-score range: {toxic_min:.4g} to {toxic_max:.4g}")
print("Comment-weighted mean by video category:")
for label in [0,1]:
    print(f"  {['Non-conspiracy','Conspiracy'][label]}: "
          f"{type_sum[label]/type_count[label] if type_count[label] else np.nan:.4f} "
          f"(n={type_count[label]:,} scored comments)")
video_tox = pd.DataFrame([{"video_id":vid,"mean_toxic":s/n,"scored_comments":n}
                          for vid,(s,n) in video_stats.items() if n])
video_tox = video_tox.merge(df[["video_id","content_type","majority_label",
                                "publish_year","Channel_ID"]], on="video_id", how="left")
display(video_tox.groupby("content_type")["mean_toxic"].agg(["count","mean","median"]).round(4))
```

```
Observed toxic-score range: 1.125e-05 to 0.9907
Comment-weighted mean by video category:
  Non-conspiracy: 0.2010 (n=118,362 scored comments)
  Conspiracy: 0.2096 (n=101,021 scored comments)
```

content_type	count	mean	median
Conspiracy	977	0.1694	0.1544
Non-conspiracy	1541	0.1359	0.1082

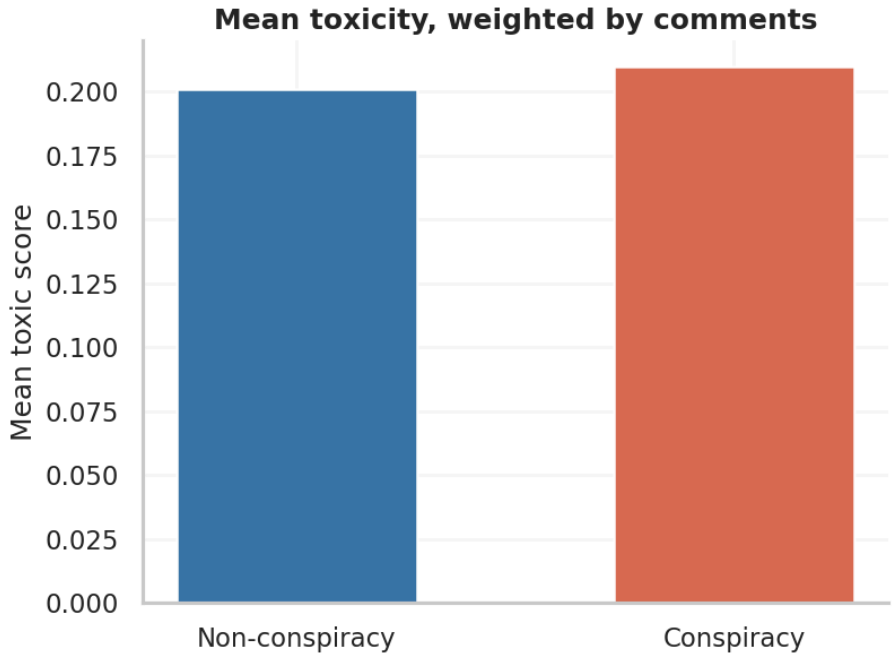
In [12]:

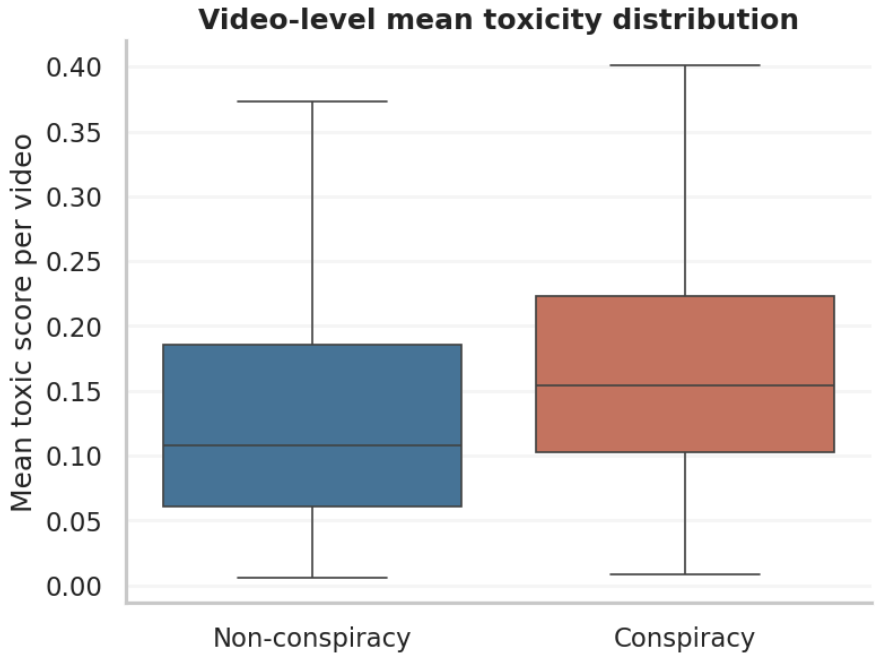
```
if not video_tox.empty:
    fig, axes = plt.subplots(1,2,figsize=(13.5,5.2))
    names = ["Non-conspiracy","Conspiracy"]
    vals = [type_sum[i]/type_count[i] if type_count[i] else np.nan for i in (0,1)]
    axes[0].bar(names, vals, color=[COLORS[n] for n in names], width=.55)
    axes[0].set(title="Mean toxicity, weighted by comments", ylabel="Mean toxic score")
    sns.boxplot(data=video_tox, x="content_type", y="mean_toxic", order=names,
                palette=[COLORS[n] for n in names], showfliers=False, ax=axes[1])
    axes[1].set(title="Video-level mean toxicity distribution", xlabel="", ylabel="Mean toxic score per video")
    finish(fig, "toxicity_video_type")
    by_year_tox = video_tox.groupby(["publish_year","content_type"])["mean_toxic"].mean().unstack()
    if not by_year_tox.empty:
        fig, ax = plt.subplots(figsize=(10,5))
        for name in names:
            if name in by_year_tox:
                ax.plot(by_year_tox.index, by_year_tox[name], marker="o",
                        label=name, color=COLORS[name], lw=2)
        ax.set(title="Mean video-level toxicity by publication year",
               xlabel="Publication year", ylabel="Mean toxicity score")
        ax.legend(frameon=False)
        finish(fig, "toxicity_by_year")
    fit_adjusted(video_tox.rename(columns={"mean_toxic":"video_mean_toxic"}),
                 "video_mean_toxic", include_channel=True)
```

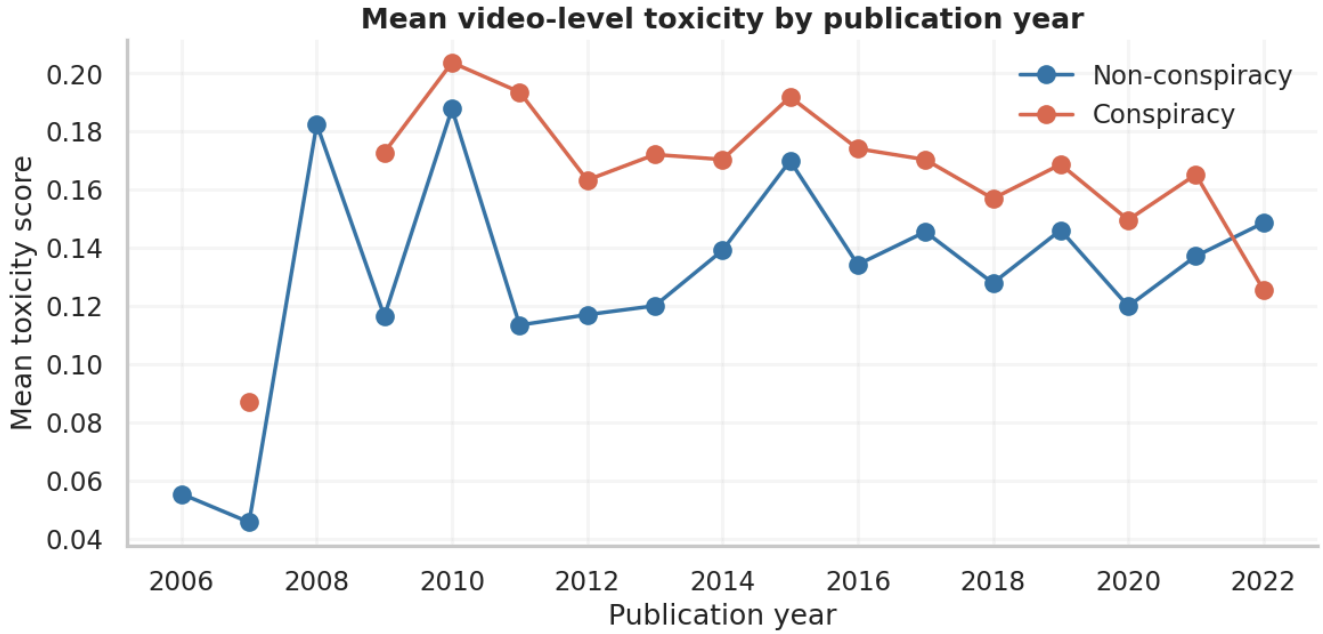
```
/tmp/ipykernel_3260809/2345312544.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=video_tox, x="content_type", y="mean_toxic", order=names,
```







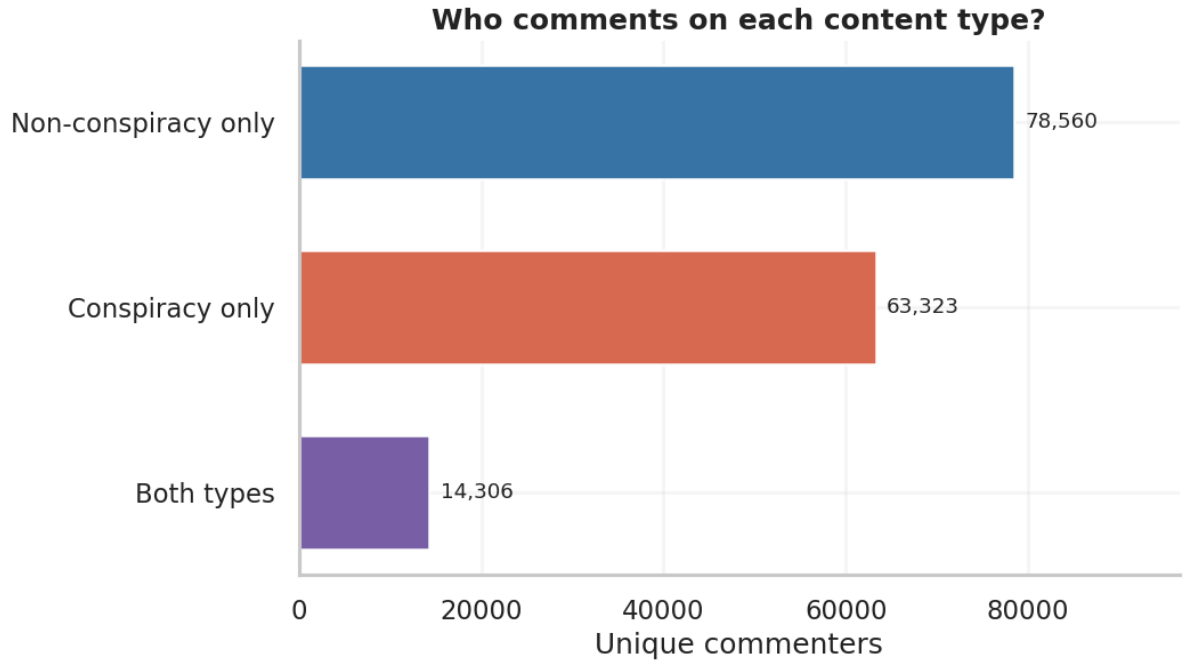
video_mean_toxic | within-channel=True | N=1,605 | channels=244 | coefficient=0.0125 | 95% CI [0.0043, 0.0207] | p=0.002905

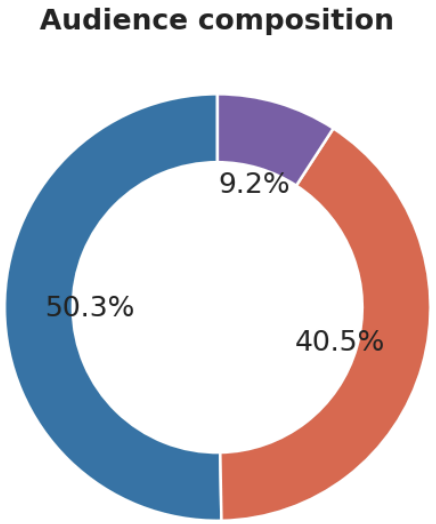
We deduplicate commenter–video pairs **globally**, not merely within individual CSV chunks. Each commenter is classified as non-conspiracy only, conspiracy only, or both. This measures observed commenting behavior, not viewing or personal identity.

In [13]:
```
parts = []
for chunk in pd.read_csv(COMMENTS_FILE, usecols=["videoId","Anon_id"],
                         dtype={"videoId":str,"Anon_id":str}, chunksize=CHUNK_SIZE):
    chunk = chunk[chunk["videoId"].isin(valid_ids) & chunk["Anon_id"].notna()]
    if not chunk.empty:
        parts.append(chunk[["Anon_id","videoId"]].drop_duplicates())
if not parts:
    raise ValueError("No commenter/video pairs matched the labeled video IDs.")
pairs = pd.concat(parts, ignore_index=True).drop_duplicates(["Anon_id","videoId"])
pairs["type"] = pairs["videoId"].map(video_type)
activity = pairs.groupby(["Anon_id","type"]).size().unstack(fill_value=0).reindex(columns=[0,1],fill_value=0)
activity["total_videos"] = activity[0] + activity[1]
activity["audience_group"] = np.select(
    [(activity[0]>0)&(activity[1]==0), (activity[1]>0)&(activity[0]==0)],
    ["Non-conspiracy only","Conspiracy only"], default="Both types")
order = ["Non-conspiracy only","Conspiracy only","Both types"]
counts = activity["audience_group"].value_counts().reindex(order,fill_value=0)
overlap = pd.DataFrame({"Commenters":counts, "Share":counts/counts.sum()})
display(overlap.style.format({"Commenters":"{:,.0f}","Share":"{:.1%}"}))
print("Total unique commenters:", f"{len(activity):,}")
print("Overlap among conspiracy commenters:",
      f"{counts['Both types']/(counts['Both types']+counts['Conspiracy only']):.1%}")
print("Overlap among non-conspiracy commenters:",
      f"{counts['Both types']/(counts['Both types']+counts['Non-conspiracy only']):.1%}")
fig, axes = plt.subplots(1,2,figsize=(13.5,5.2),gridspec_kw={"width_ratios":[1.5,1]})
bar_colors = [COLORS["Non-conspiracy"],COLORS["Conspiracy"],COLORS["Both types"]]
bars = axes[0].barh(order[::-1], counts.loc[order[::-1]], color=bar_colors[::-1],height=.62)
for bar in bars:
    axes[0].text(bar.get_width()+max(counts)*.015,bar.get_y()+bar.get_height()/2,
                 f"{bar.get_width():,.0f}",va="center",fontsize=11)
axes[0].set(xlim=(0,max(counts)*1.23),xlabel="Unique commenters",title="Who comments on each content type?")
axes[1].pie(counts, labels=None, colors=bar_colors, startangle=90,
            wedgeprops={"width":.32,"edgecolor":"white"},
            autopct=lambda p:f"{p:.1f}%" if p>4 else "")
axes[1].set(title="Audience composition")
finish(fig,"audience_overlap")
```

audience_group	Commenters	Share
Non-conspiracy only	78,560	50.3%
Conspiracy only	63,323	40.5%
Both types	14,306	9.2%

```
Total unique commenters: 156,189
Overlap among conspiracy commenters: 18.4%
Overlap among non-conspiracy commenters: 15.4%
```



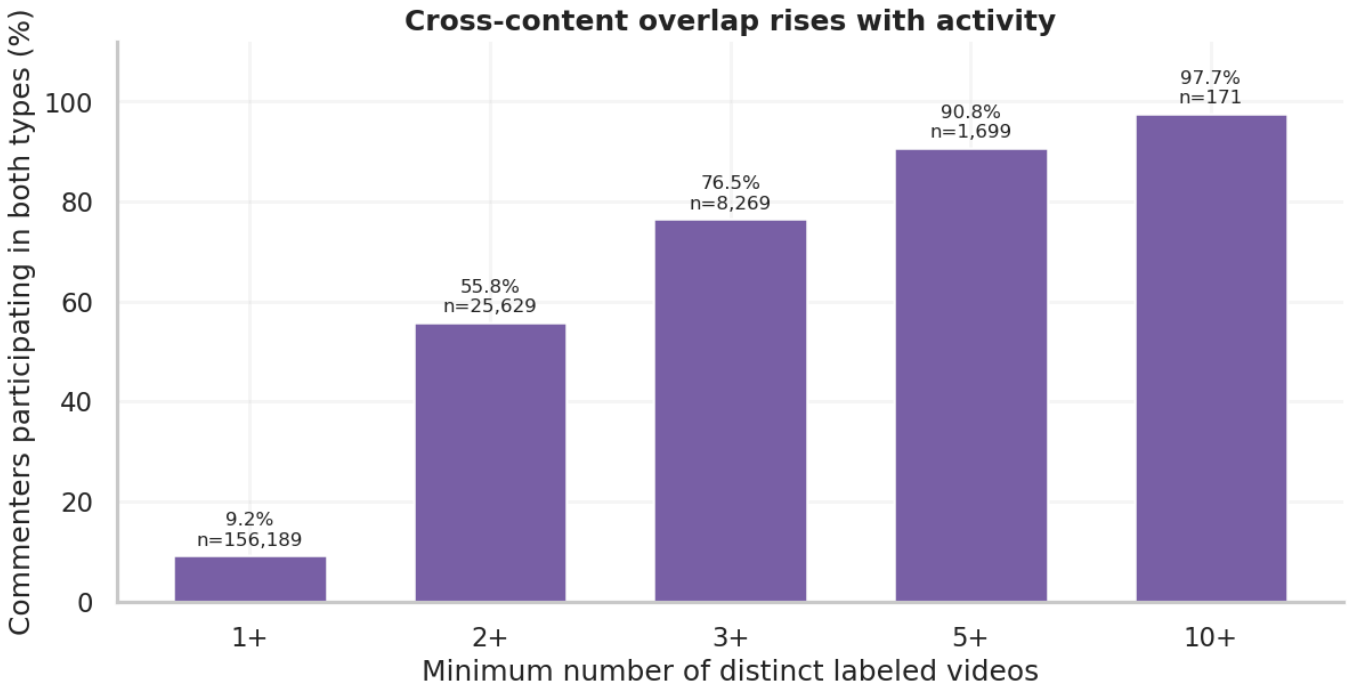


9 | Overlap by participation intensity

Thresholds count distinct labeled videos, not raw comments. Higher overlap is partly mechanical: commenting on more videos creates more opportunities to encounter both types.

In [14]:
```
thresholds = [1,2,3,5,10]
activity_rows = []
for minimum in thresholds:
    subset = activity[activity["total_videos"]>=minimum]
    activity_rows.append({"Minimum distinct videos":minimum, "Commenters":len(subset),
                          "Both types":int((subset["audience_group"]=="Both types").sum()),
                          "Overlap share":(subset["audience_group"]=="Both types").mean()})
activity_summary = pd.DataFrame(activity_rows)
display(activity_summary.style.format({"Commenters":"{:,.0f}","Both types":"{:,.0f}",
                                       "Overlap share":"{:.1%}"}))
fig, ax = plt.subplots(figsize=(10.2,5.4))
x = np.arange(len(activity_summary))
bars = ax.bar(x,activity_summary["Overlap share"]*100,color="#7A61A8",width=.64)
for bar, n in zip(bars,activity_summary["Commenters"]):
    ax.text(bar.get_x()+bar.get_width()/2,bar.get_height()+2,
            f"{bar.get_height():.1f}%\nn={n:,}",ha="center",fontsize=10)
ax.set(xticks=x,xticklabels=[f"{n}+" for n in thresholds],ylim=(0,112),
       xlabel="Minimum number of distinct labeled videos",
       ylabel="Commenters participating in both types (%)",
       title="Cross-content overlap rises with activity")
finish(fig,"overlap_by_activity")
```

	Minimum distinct videos	Commenters	Both types	Overlap share
0	1	156,189	14,306	9.2%
1	2	25,629	14,306	55.8%
2	3	8,269	6,329	76.5%
3	5	1,699	1,542	90.8%
4	10	171	167	97.7%

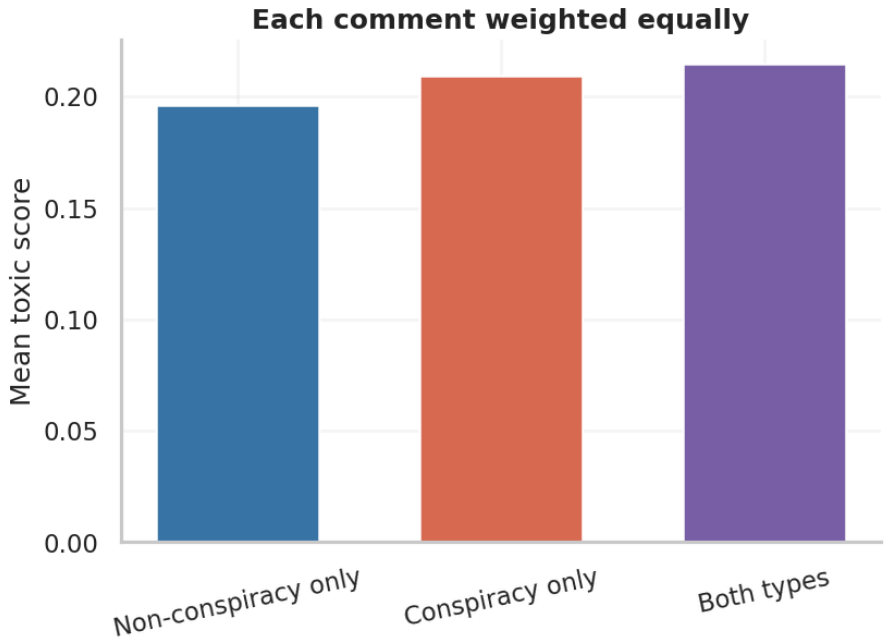


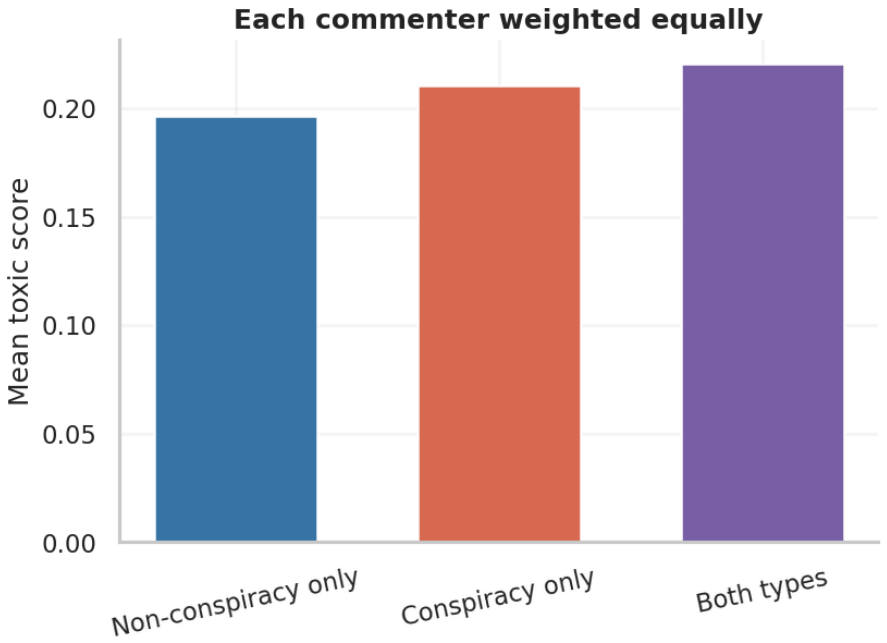
10 | Toxicity by commenter audience group

Two estimands are shown: (1) **comment-weighted mean**, giving each scored comment equal weight, and (2) **commenter-weighted mean**, giving each commenter equal weight. Comparisons may reflect differences in activity and video selection.

In [15]:
```
audience_lookup = activity["audience_group"].to_dict()
agg_parts = []
for chunk in pd.read_csv(COMMENTS_FILE, usecols=["videoId","Anon_id","toxic"],
                         dtype={"videoId":str,"Anon_id":str}, chunksize=CHUNK_SIZE):
    chunk = chunk[chunk["videoId"].isin(valid_ids) & chunk["Anon_id"].isin(activity.index)].copy()
    chunk["toxic"] = pd.to_numeric(chunk["toxic"], errors="coerce")
    chunk = chunk.dropna(subset=["toxic"])
    if chunk.empty: continue
    grouped = chunk.groupby("Anon_id")["toxic"].agg(["sum","count"])
    agg_parts.append(grouped)
if not agg_parts:
    raise ValueError("No valid toxicity scores for matched commenters.")
commenter_totals = pd.concat(agg_parts).groupby(level=0).sum()
commenter_totals["mean_score"] = commenter_totals["sum"]/commenter_totals["count"]
commenter_totals["audience_group"] = commenter_totals.index.map(audience_lookup)
commenter_totals = commenter_totals.join(activity[["total_videos"]])
grouped = commenter_totals.groupby("audience_group").agg(
    commenters=("mean_score","size"), scored_comments=("count","sum"),
    toxicity_sum=("sum","sum"), commenter_weighted_mean=("mean_score","mean"))
grouped["comment_weighted_mean"] = grouped["toxicity_sum"]/grouped["scored_comments"]
grouped = grouped.reindex(order)
display(grouped[["commenters","scored_comments","comment_weighted_mean","commenter_weighted_mean"]].round(4))
fig, axes = plt.subplots(1,2,figsize=(14,5.3))
for ax,(metric,title) in zip(axes,[("comment_weighted_mean","Each comment weighted equally"),
                                   ("commenter_weighted_mean","Each commenter weighted equally")]):
    ax.bar(order,grouped[metric],color=bar_colors,width=.62)
    ax.set(title=title,ylabel="Mean toxic score")
    ax.tick_params(axis="x",labelrotation=12)
finish(fig,"toxicity_by_audience")
```

audience_group	commenters	scored_comments	comment_weighted_mean	commenter_weighted_mean
Non-conspiracy only	76501	94102	0.1963	0.1965
Conspiracy only	61807	77146	0.2095	0.2103
Both types	14194	48135	0.2149	0.2207



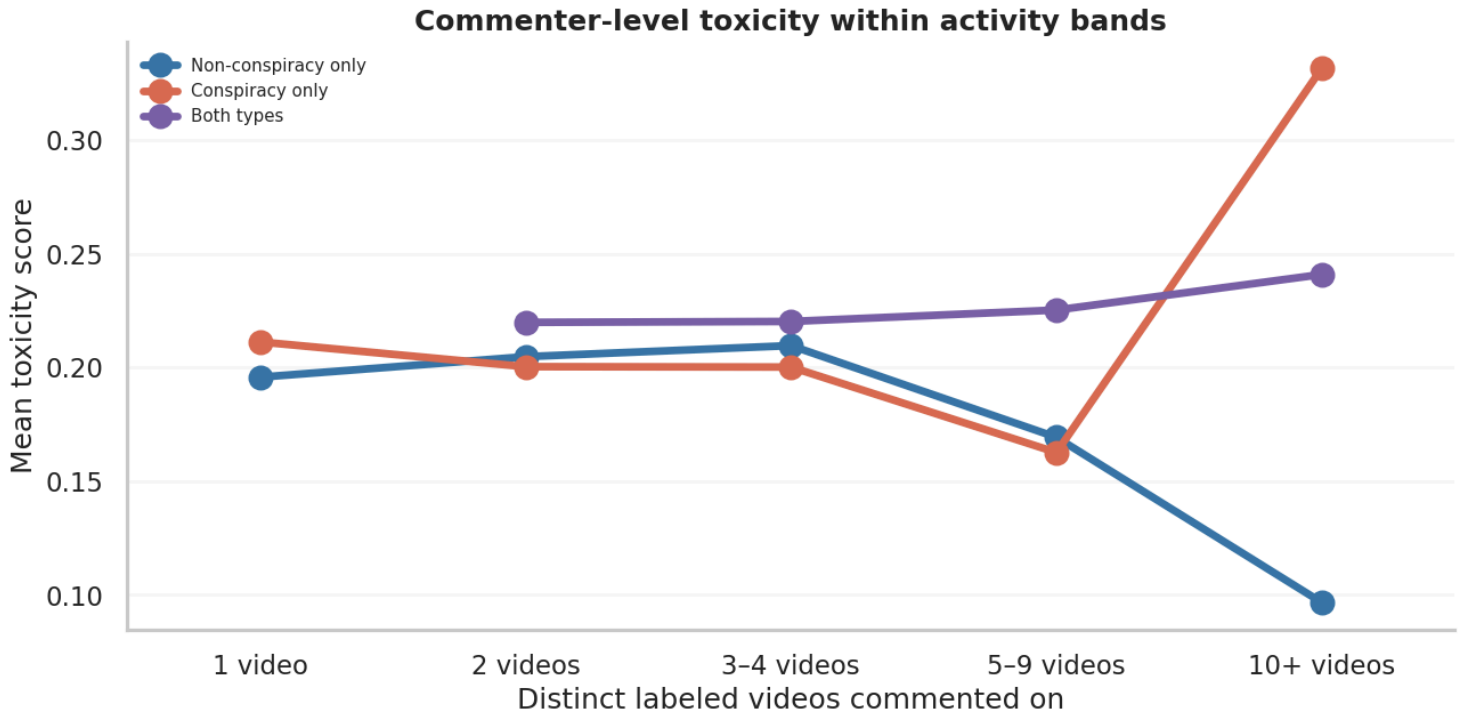


11 | Activity-adjusted commenter toxicity

We compare audience groups within participation bands. These comparisons remain observational; the 'both types' group cannot exist among commenters observed on only one distinct video.

In [16]:
```
bins = [0,1,2,4,9,np.inf]
band_labels = ["1 video","2 videos","3–4 videos","5–9 videos","10+ videos"]
commenter_totals["activity_band"] = pd.cut(commenter_totals["total_videos"],bins=bins,labels=band_labels)
band_summary = (commenter_totals.groupby(["activity_band","audience_group"],observed=True)
                .agg(commenters=("mean_score","size"),mean_toxicity=("mean_score","mean"))
                .reset_index())
display(band_summary)
fig,ax = plt.subplots(figsize=(11,5.6))
sns.pointplot(data=band_summary,x="activity_band",y="mean_toxicity",hue="audience_group",
              hue_order=order,palette=bar_colors,markers="o",linestyles="-",ax=ax,
              errorbar=None)
ax.set(title="Commenter-level toxicity within activity bands",
       xlabel="Distinct labeled videos commented on",ylabel="Mean toxicity score")
ax.legend(title="",fontsize=9,frameon=False)
finish(fig,"toxicity_activity_bands")
```

	activity_band	audience_group	commenters	mean_toxicity
0	1 video	Conspiracy only	57165	0.211187
1	1 video	Non-conspiracy only	69938	0.195708
2	2 videos	Both types	7903	0.219798
3	2 videos	Conspiracy only	3888	0.200318
4	2 videos	Non-conspiracy only	5397	0.204711
5	3–4 videos	Both types	4756	0.220187
6	3–4 videos	Conspiracy only	696	0.200151
7	3–4 videos	Non-conspiracy only	1070	0.209577
8	5–9 videos	Both types	1369	0.225211
9	5–9 videos	Conspiracy only	57	0.162611
10	5–9 videos	Non-conspiracy only	93	0.169272
11	10+ videos	Both types	166	0.240852
12	10+ videos	Conspiracy only	1	0.331466
13	10+ videos	Non-conspiracy only	3	0.096657



12 | Within-commenter comparison for cross-content participants

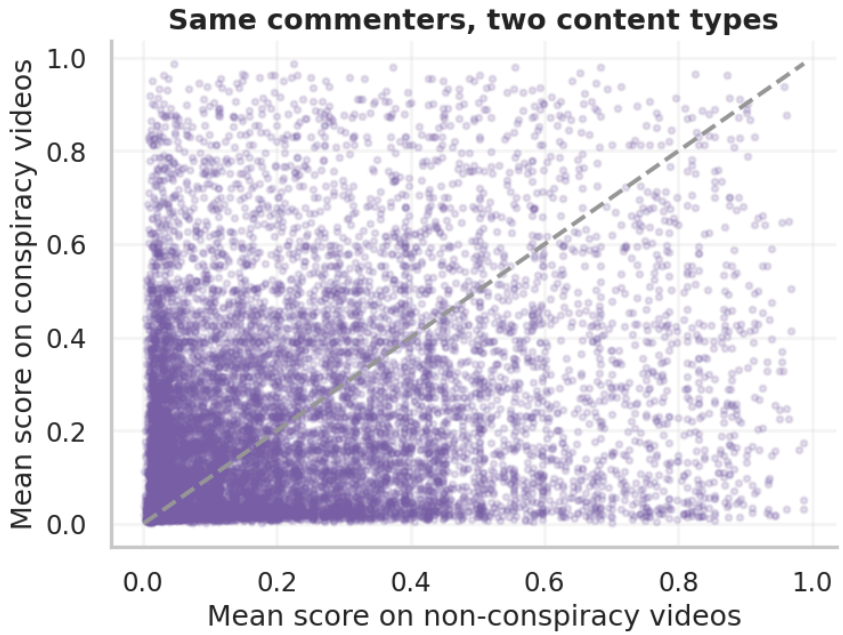
For commenters active in both categories, compare their average toxicity **within the same commenter** across the two video types. This controls for stable commenter-level differences descriptively, but video/topic context and unequal comment counts remain confounders.

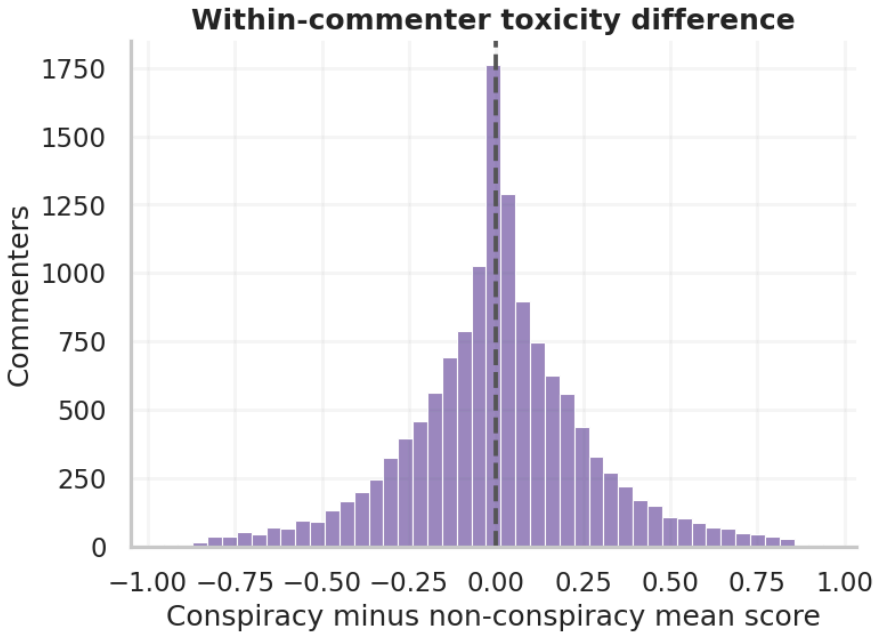
In [17]:
```
both_ids = set(activity.index[activity["audience_group"]=="Both types"])
within_parts = []
```

```python
for chunk in pd.read_csv(COMMENTS_FILE,usecols=["videoId","Anon_id","toxic"],
                         dtype={"videoId":str,"Anon_id":str},chunksize=CHUNK_SIZE):
    chunk = chunk[chunk["videoId"].isin(valid_ids) & chunk["Anon_id"].isin(both_ids)].copy()
    chunk["toxic"] = pd.to_numeric(chunk["toxic"],errors="coerce")
    chunk = chunk.dropna(subset=["toxic"])
    if chunk.empty: continue
    chunk["type"] = chunk["videoId"].map(video_type)
    within_parts.append(chunk.groupby(["Anon_id","type"])["toxic"].agg(["sum","count"]))
if within_parts:
    within = pd.concat(within_parts).groupby(level=[0,1]).sum()
    within["mean"] = within["sum"]/within["count"]
    paired = within["mean"].unstack().dropna(subset=[0,1])
    paired["difference"] = paired[1]-paired[0]
    print(f"Cross-content commenters with scored comments in both categories: {len(paired):,}")
    display(paired[[0,1,"difference"]].describe().round(4))
    fig,axes = plt.subplots(1,2,figsize=(13,5))
    axes[0].scatter(paired[0],paired[1],s=10,alpha=.17,color=COLORS["Both types"],rasterized=True)
    low = min(paired[0].min(),paired[1].min()); high = max(paired[0].max(),paired[1].max())
    axes[0].plot([low,high],[low,high],"--",color="#9A9A9A")
    axes[0].set(xlabel="Mean score on non-conspiracy videos",
                ylabel="Mean score on conspiracy videos",
                title="Same commenters, two content types")
    sns.histplot(paired["difference"],bins=45,color=COLORS["Both types"],ax=axes[1])
    axes[1].axvline(0,ls="--",color="#555")
    axes[1].set(xlabel="Conspiracy minus non-conspiracy mean score",
                ylabel="Commenters",title="Within-commenter toxicity difference")
    finish(fig,"within_commenter_toxicity")
else:
    print("No matched scored comments for cross-content participants.")
```

```
Cross-content commenters with scored comments in both categories: 13,535
```

type	0	1	difference
count	13535.0000	13535.0000	13535.0000
mean	0.2187	0.2225	0.0039
std	0.2106	0.2134	0.2606
min	0.0014	0.0024	-0.9551
25%	0.0465	0.0508	-0.1287
50%	0.1516	0.1548	0.0026
75%	0.3276	0.3300	0.1383
max	0.9875	0.9863	0.9398





13 | Benchmark checks, interpretation & limitations

Previously reported overlap counts are reference values only; all calculations above use the actual files. Differences can reflect changes in inputs, missing IDs, label handling, or duplicate treatment.

In [18]:
```
expected = {"Non-conspiracy only":78560,"Conspiracy only":63323,"Both types":14306}
check = pd.DataFrame({"Reported":[expected[k] for k in order],
                      "Computed":[int(counts[k]) for k in order]},index=order)
check["Difference"] = check["Computed"]-check["Reported"]
display(check)
expected_activity = {1:(156189,9.2),2:(25629,55.8),3:(8269,76.5),5:(1699,90.8),10:(171,97.7)}
checks = []
for _,r in activity_summary.iterrows():
    t=int(r["Minimum distinct videos"])
    n,p=expected_activity[t]
    checks.append({"Threshold":t,"Reported commenters":n,"Computed commenters":int(r["Commenters"]),
                   "Reported overlap (%)":p,"Computed overlap (%)":100*r["Overlap share"]})
display(pd.DataFrame(checks).round(2))
```

	Reported	Computed	Difference
Non-conspiracy only	78560	78560	0
Conspiracy only	63323	63323	0
Both types	14306	14306	0

	Threshold	Reported commenters	Computed commenters	Reported overlap (%)	Computed overlap (%)
0	1	156189	156189	9.2	9.16
1	2	25629	25629	55.8	55.82
2	3	8269	8269	76.5	76.54
3	5	1699	1699	90.8	90.76
4	10	171	171	97.7	97.66